

Maximum Number of Non-overlapping Palindrome Substrings

LeetCode 2472 · Hard

Dynamic Programming

Strings

Palindrome

Interval Scheduling DP

1. Problem

Given a string `s` and a positive integer `k`, pick a set of **non-overlapping** substrings of `s` such that every chosen substring:

- has length $\geq k$, and
- is a palindrome.

Return the **maximum count** of substrings you can pick.

Constraint	Value
Length of <code>s</code>	$1 \leq s.length \leq 2000$
<code>k</code>	$1 \leq k \leq s.length$
Return	An integer — the maximum number of valid non-overlapping palindromic substrings

Important observations from the statement

- We only care about the **count** of substrings picked, not which characters are "left over" — leftover characters can simply be skipped.
- "Non-overlapping" means once a substring occupies range `[i, j]`, no other chosen substring can touch any index inside `[i, j]`.
- Longer palindromes are never individually "worth more" — one length-10 palindrome still counts as 1. This hints that **shorter valid palindromes are always at least as good** as longer ones, because they free up more room for future picks.

2. What Are We Really Being Asked?

Strip away the palindrome flavor for a moment. We are given a line of `n` characters, and a collection of "candidate intervals" — every substring range `[i, j]` that happens to be a palindrome of length $\geq k$. From these candidate intervals we must pick as many as possible such that **no two picked intervals share an index**.

This is exactly the classic **weighted interval scheduling** problem, except every interval has weight 1 (we're maximizing count, not sum), and the "candidate intervals" are generated by a palindrome check instead of being handed to us directly.

Given

String `s`, threshold length `k`

Changing	Which substrings are chosen; where the "cursor" is as we scan left to right
To optimize	Maximize the <i>count</i> of chosen non-overlapping palindromic substrings of length $\geq k$
What makes it hard	Two separate sub-problems are tangled together: (a) knowing which ranges are palindromes at all, and (b) choosing a non-overlapping subset of those ranges optimally

3. Key Observations

Observation 1 — Palindrome checking has overlapping sub-problems

$s[i..j]$ is a palindrome only if $s[i] == s[j]$ and $s[i+1..j-1]$ is a palindrome. **Why it's true:** stripping the two outer matching characters from a palindrome always leaves a smaller palindrome (or an empty/1-length string). **Why it helps:** it means "is this range a palindrome" can be computed once for every range in $O(n^2)$ total time by building from shorter ranges to longer ranges, instead of re-scanning each range from scratch (which would cost $O(n^3)$).

Observation 2 — Greedy "pick shortest palindrome" alone isn't obviously safe, but DP over prefixes is

You might be tempted to greedily grab the first valid palindrome you see. It *happens* to work for this particular problem (there's a known two-pointer greedy), but it needs a separate correctness proof. **Why a DP is helpful instead:** if we define $dp[i]$ = best answer using only the prefix $s[0..i-1]$, then at every position we only have two honest choices — extend the answer without using this final position, or close off a palindrome that ends exactly here. Trying every legal closing point removes the need to prove a greedy rule at all.

Observation 3 — A chosen substring "resets" the problem

If a palindrome occupies $s[j..i-1]$, then whatever happens inside $[j, i-1]$ is irrelevant to future picks — the next chosen substring must start at index i or later. **Why it helps:** this is exactly what lets us write $dp[j] + 1$ as a candidate value for $dp[i]$ — $dp[j]$ already represents "the best we can do using everything strictly before this palindrome," so there's no double-counting or overlap to worry about.

4. Intuition

Walk through the string left to right, one prefix length at a time. At every prefix length i , ask one question: "What is the best answer I can get using only the first i characters?"

There are exactly two ways to answer this:

1. **Character $i-1$ is not the end of any chosen palindrome.** Then the best answer is simply whatever the best answer was one character ago — $dp[i-1]$.

2. Character `i-1` is the last character of some chosen palindrome `s[j..i-1]`. Then the best answer is "the best we could do before this palindrome started" plus one for this palindrome itself — `dp[j] + 1`, provided `s[j..i-1]` is actually a palindrome of length $\geq k$.

Since we don't know in advance which `j` gives the best palindrome ending at `i-1`, we simply try every legal `j` (from `0` up to `i-k`, since the palindrome needs length $\geq k$) and keep the best. This is only correct because we've already pre-computed, for free, whether every `[j, i-1]` range is a palindrome — that's Observation 1 paying off.

One-line mental model: "Sweep a finish line across the string. At each finish line, either nothing new ends here, or some palindrome ends here — and if it does, add 1 to whatever the best score was right before it started."

5. Approach — Palindrome Precomputation + Prefix DP

Only one approach was provided in the source code — a baseline/brute-force is not covered separately since none was supplied. This is the approach actually used.

5.1 Idea

Two stages:

1. **Stage A — Palindrome table.** Build a 2D boolean table `pal[i][j]` that answers "is `s[i..j]` a palindrome?" for every `(i, j)` pair, using the expand-from-smaller-ranges recurrence from Observation 1.
2. **Stage B — Prefix DP.** Build `dp[0..n]` where `dp[i]` = max non-overlapping valid palindromes using only `s[0..i-1]`, using the two-choice recurrence from the Intuition section.

5.2 Why it works

Stage A guarantees `pal[j][i-1]` is a true/false answer, computed correctly, in `O(1)` lookup time by the time Stage B needs it. Stage B is a standard "take or skip at each position" DP identical in spirit to weighted interval scheduling, so trying every candidate `j` guarantees the true optimum is never missed (see Section 10 for the full correctness argument).

5.3 Pseudocode

```
// Stage A: pal[i][j] = true if s[i..j] is a palindrome
for len = 1..n:
    for i such that i+len-1 < n:
        j = i+len-1
        if len == 1:      pal[i][j] = true
        elif len == 2:    pal[i][j] = (s[i]==s[j])
        else:             pal[i][j] = (s[i]==s[j]) AND pal[i+1][j-1]

// Stage B: dp[i] = best count using prefix s[0..i-1]
dp[0] = 0
for i = 1..n:
    dp[i] = dp[i-1]                // choice 1: skip position i-1
    for j = 0..(i-k):             // choice 2: close a palindrome s[j..i-1]
        if pal[j][i-1]:
            dp[i] = max(dp[i], dp[j] + 1)
```

```

        dp[i] = max(dp[i], dp[j] + 1)
    return dp[n]

```

5.4 Visual — filling the palindrome table

Ranges are filled diagonally, shortest to longest. Each cell depends only on the cell one step diagonally inward:

	len 1	len 2	len 3	...
depends on	nothing	<code>s[i]==s[j]</code>	<code>s[i]==s[j] & pal[i+1][j-1]</code>	same pattern

This guarantees every shorter range needed by a longer range is already computed — the classic bottom-up interval-DP fill order.

5.5 Time and space complexity

Stage	Time	Space	Notes
A — palindrome table	$O(n^2)$	$O(n^2)$	n^2 ranges, $O(1)$ work each
B — prefix DP	$O(n^2)$	$O(n)$	outer loop n , inner loop up to n
Total	$O(n^2)$	$O(n^2)$	space dominated by the palindrome table; with $n \leq 2000$ this is well within limits

5.6 Implementation (as provided, unmodified — no bugs found)

```

int maxPalindromes(string s, int k) {
    int n = s.size();

    // pal[i][j] = true if s[i...j] is a palindrome
    vector<vector<bool>> pal(n, vector<bool>(n, false));

    for (int len = 1; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;

            if (len == 1)
                pal[i][j] = true;
            else if (len == 2)
                pal[i][j] = (s[i] == s[j]);
            else
                pal[i][j] = (s[i] == s[j] && pal[i + 1][j - 1]);
        }
    }

    // dp[i] = maximum number of non-overlapping palindromes
    // using first i characters
    vector<int> dp(n + 1, 0);

    for (int i = 1; i <= n; i++) {

```

```

// Don't take a palindrome ending at i-1
dp[i] = dp[i - 1];

// Try every palindrome ending at i-1
for (int j = 0; j <= i - k; j++) {
    if (pal[j][i - 1]) {
        dp[i] = max(dp[i], dp[j] + 1);
    }
}

return dp[n];
}

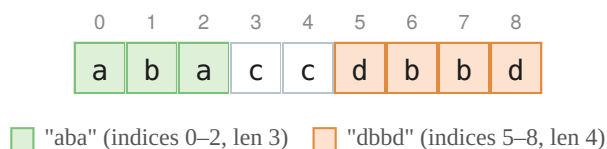
```

Design note: using a DP array of size `n+1` indexed by *prefix length* (rather than the more common size- `n` array indexed by ending position) is a clean choice — it avoids the awkward `j == 0 ? 0 : dp[j-1]` special-casing that many other accepted solutions to this problem need for palindromes starting at index 0.

6. Detailed Dry Run

Test case: `s = "abacdbbd"`, `k = 3` (expected output: 2)

6.1 The string, indexed



6.2 Relevant palindrome table entries (only cells that matter for this run)

Range	Substring	pal[i][j]	Why
pal[0][2]	"aba"	true	s[0]==s[2]=='a', pal[1][1] (single char) = true
pal[3][4]	"cc"	true	s[3]==s[4]=='c' — but length 2 < k, never used in Stage B
pal[6][7]	"bb"	true	s[6]==s[7]=='b' — length 2 < k, only used as an inner check
pal[5][8]	"dbbd"	true	s[5]==s[8]=='d', and inner pal[6][7] ("bb") = true
everything else length ≥ 3	—	false	outer characters never match

6.3 DP array progression (k = 3)

i (prefix length)	dp[i]	What happened
0	0	base case — empty prefix
1, 2	0, 0	too short for any length-3 palindrome to close here yet
3	1	j=0 works: pal[0][2] ("aba") is true → dp[3] = dp[0]+1 = 1

4, 5, 6, 7, 8	1, 1, 1, 1, 1	no new palindrome closes at any of these positions → carry $dp[i-1]$ forward
9	2	$j=5$ works: $pal[5][8]$ ("dbbd") is true → $dp[9] = dp[5] + 1 = 1 + 1 = 2$

Final answer: $dp[9] = 2$ — matches the expected output. The two palindromes picked are "aba" (closing $dp[3]$) and "dbbd" (closing $dp[9]$, built on top of $dp[5]$, which still equals $dp[3]$ since nothing new happened in between).

7. Test Cases

Case	Input	Output	Explanation
Normal case	$s = \text{"abaccdbbd"}, k=3$	2	"aba" and "dbbd" are disjoint valid palindromes (traced above)
No valid palindrome	$s = \text{"adbcda"}, k=2$	0	No adjacent pair matches, so no palindrome of length ≥ 2 exists at all
Smallest valid case	$s = \text{"a"}, k=1$	1	A single character is always a palindrome of length 1
Overlap-avoidance check	$s = \text{"aaaa"}, k=2$	2	Greedily grabbing "aaaa" (len 4) as one palindrome would give only 1; the DP correctly finds two disjoint length-2 palindromes ("aa" + "aa") instead — this is exactly why shorter palindromes matter (Observation in §1)
k equals string length	$s = \text{"aba"}, k=3$	1	Only one palindrome (the whole string) can possibly qualify; no room for a second

8. Why the Solution Works

The correctness argument has two independent parts, matching the two stages:

Palindrome table is exhaustive and correct

Every range is classified using only strictly shorter, already-verified ranges, and the recurrence ($s[i] == s[j]$ and inner range is a palindrome) is a necessary and sufficient condition for a range to be a palindrome. No range is ever skipped, since the double loop covers every (i, j) pair with $i \leq j$.

The DP considers every possible "last decision"

For any prefix of length i , the optimal solution either does or does not use character $i-1$ as the end of a chosen palindrome — there is no third option. The code checks both: carrying forward $dp[i-1]$ covers "does not end here," and the inner loop over every legal j covers "ends here, starting at j ," for every

possible starting point. Because `dp[j]` already represents the optimal answer using everything strictly before index `j`, adding 1 never double-counts or overlaps with the new palindrome.

Since both stages are exhaustive and the DP never discards a valid candidate, `dp[n]` is guaranteed to be the true optimum.

9. Common Mistakes

Mistake	Why it's wrong
Only checking palindromes of length exactly <code>k</code>	The condition is <code>length ≥ k</code> — longer valid palindromes must also be considered, otherwise valid closing points are missed
Greedily taking the <i>first</i> palindrome found without justification	Works for this specific problem structure but is not obviously correct in general — must be proven, or just avoided by using the DP
Off-by-one in the inner loop bound (<code>j ≤ i - k</code>)	Using <code>j < i - k</code> or <code>j ≤ i - k - 1</code> silently excludes the shortest valid palindromes ending at <code>i - 1</code>
Forgetting the length-2 base case in the palindrome table	The general recurrence <code>pal[i+1][j-1]</code> is undefined/out-of-bounds when <code>j - 1 < i + 1</code> , i.e. exactly when length is 2 — it needs its own base case
Indexing <code>dp</code> by ending position instead of prefix length without care	Leads to awkward <code>j == 0 ? 0 : dp[j - 1]</code> special-casing and is an easy source of off-by-one bugs (this code avoids it — see design note in §5)
Assuming $O(n^3)$ brute-force palindrome checking is fine	With <code>n</code> up to 2000, $O(n^3)$ ($\approx 8 \times 10^9$) would TLE; the $O(n^2)$ table build is required

10. Pattern Recognition

How to recognize this type of problem again:

- Phrases like "*maximum number of non-overlapping...*" or "*select a set of disjoint substrings/intervals...*" → think **interval scheduling DP**: `dp[i] = max(dp[i-1], max over valid j of dp[j] + value)`.
- Phrases like "*substring is a palindrome*" combined with needing to check many ranges → think **palindrome-table precomputation** ($O(n^2)$ DP) before touching the main DP.
- A DP where the "state" is a **prefix of the array/string** and the transition asks "what happened at the very last position" → a strong signal to define `dp[i]` over prefix length rather than a single index, to cleanly avoid boundary special-casing.
- When a problem's answer only cares about *count*, not *which exact objects*, and the objects are ranges over a sequence → almost always reduces to weighted (or unweighted) interval scheduling.

11. Final Takeaway

CORE IDEA: Precompute which ranges are palindromes, then run a prefix DP that at each position either skips it or closes off a valid palindrome ending there.

RECURRENCE:

```
pal[i][j] = (s[i]==s[j]) && (len≤2 || pal[i+1][j-1])
```

```
dp[i] = max(dp[i-1], max over j≤i-k with pal[j][i-1] of dp[j]+1)
```

KEY OBSERVATION: Once a palindrome closes at some position, everything before its start is fully summarized by `dp[j]` — no overlap bookkeeping needed.

COMPLEXITY: Time $O(n^2)$, Space $O(n^2)$ (dominated by the palindrome table).

PATTERN: Palindrome precomputation + prefix-based interval scheduling DP.